SOFTWARE ENGINEERING METHODOLOGIES & SCRUM

METHODOLOGICAL FOUNDATIONS & PARADIGMS

1.1 The Philosophi of Software Development

Software engineering differs fundamentally from classical engineering disciplines such as civil or mechanical engineering. In traditional civil engineering, physical materials behave according to established physical laws, and the primary constraints are governed by statics, dynamics, and material science. Once a bridge or building design is validated through structural mechanics, the execution phase focuses on precise physical replication.
In contrast, software construction is an entirely intellectual and invisible process. Software systems are complex, highly malleable, and non-linear. The cost of altering a software design is drastically lower than the cost of altering a physical structure, leading stakeholders to expect continuous modifications throughout the lifecycle.

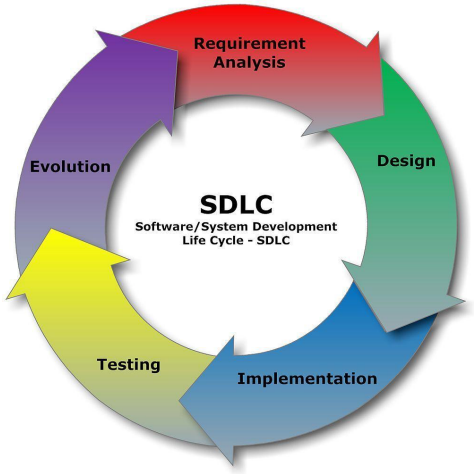
Software development relies on two distinct process control models:

- **Defined Process Control:** Assumes that the underlying production process is fully understood, highly predictable, and repeatable. If the input parameters are tightly controlled, the output will consistently meet specified standards without major adjustments.
- **Empirical Process Control:** Assumes that the underlying process is complex, partially unmapped, and subject to high rates of unexpected change. Instead of relying on upfront planning, empirical control relies on continuous observation, measurement, and adaptation.

Traditional software frameworks operate on the premise that software can be governed by defined process control. Modern software methodologies recognize that software design is an empirical problem, where system requirements, platform capabilities, and market demands change unpredictably during production.

1.2 Failure of Plan-Driven Models

Plan-driven methodologies, primarily exemplified by the **Waterfall Model**, divide the software development lifecycle (SDLC) into sequential phases:



Each stage must be fully completed, documented, and signed off before the next phase begins. While this model offers structured predictability for low-complexity or safety-critical domains with frozen specifications, it exhibits systemic failure modes when applied to dynamic environments.

The Requirement Volatility Problem

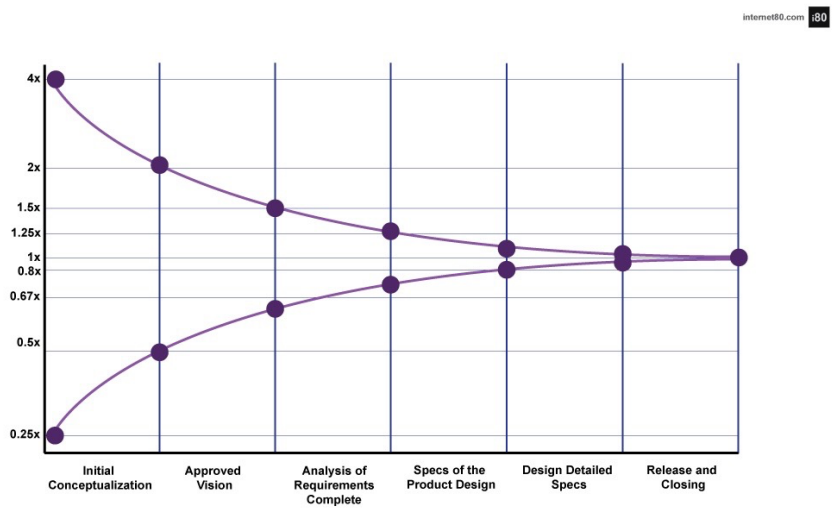
Plan-driven models assume that system requirements can be completely discovered, articulated, and locked down during the initial phase. In real-world software, requirements continuously evolve because:

- Stakeholders rarely understand their actual operational needs until they interact with a working application interface.
- External market dynamics, legal regulations, and competitor movements shift business priorities mid-project.
- Technical discoveries during implementation reveal unpredicted constraints or architectural dead ends.

Attempting to freeze requirements in a dynamic environment creates a widening gap between what the project plan delivers and what the business actually needs.

Boehm's Cone of Uncertainty

In the early phases of a project, our knowledge about the final effort, schedule, and architectural details is naturally limited. Barry Boehm formalized this phenomenon through the **Cone of Uncertainty**



At the initial concept stage, estimation variability can range by a factor of 4x to 0.25x. As development progresses, implementation details emerge, and real working code is produced, the variance narrows toward 1.0x.
Plan-driven frameworks commit to fixed budgets, schedules, and delivery dates at the start of the project—the exact point when uncertainty is at its peak.

The Risk Accumulation Profile

In a sequential Waterfall model, testing and integration occur late in the cycle. Consequently, operational risk accumulates silently throughout development and peaks during the late testing phase.

If structural architectural defects, security vulnerabilities, or core integration failures are discovered late in testing, fixing them requires reworking design, code, and documentation across the entire stack—leading to severe budget and schedule overruns.

1.3 The Emergence of the Agile Paradigm

In February 2001, seventeen software practitioners met in Snowbird, Utah, to address the systemic failures of heavy, process-centric development methodologies. The outcome was the **Manifesto for Agile Software Development**, establishing four core values:

1. **Individuals and interactions over** Processes and tools
2. **Working software over** Comprehensive documentation
3. **Customer collaboration over** Contract negotiation
4. **Responding to change over** Following a plan

These values do not discard processes, documentation, contracts, or planning. Instead, they prioritize continuous adaptability and working software over static administrative overhead.

The 12 Principles of Agile Software

The Agile Manifesto is supported by twelve operational principles designed to guide software delivery:

1. **Customer Satisfaction:** Highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. **Welcome Changing Requirements:** Harness change for the customer's competitive advantage, even late in development.
3. **Frequent Delivery:** Deliver working software frequently, from a couple of weeks to a couple of months, preferring shorter timescales.
4. **Cross-Functional Collaboration:** Business people and developers must work together daily throughout the project.
5. **Motivated Individuals:** Build projects around motivated individuals; give them the environment and support they need, and trust them to get the job done.
6. **Face-to-Face Communication:** The most efficient and effective method of conveying information is face-to-face conversation.
7. **Working Software as Progress:** Working software is the primary measure of progress.
8. **Sustainable Development:** Promoters, developers, and users should be able to maintain a constant pace indefinitely.
9. **Technical Excellence:** Continuous attention to technical excellence and good design enhances agility.
10. **Simplicity:** The art of maximizing the amount of work not done is essential.
11. **Self-Organizing Teams:** The best architectures, requirements, and designs emerge from self-organizing teams.
12. **Regular Reflection:** At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

1.4 Trade-off Matrix: Plan-Driven vs. Agile Methods

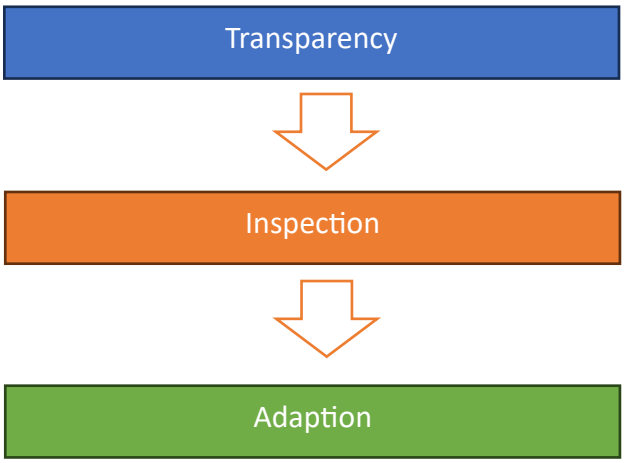
Choosing between plan-driven and agile frameworks requires evaluating organizational context, technical constraints, and risk tolerances.

Architectural Vector	Traditional Plan-Driven (Waterfall)	Agile Frameworks (Scrum, Kanban)
Core Assumption	System requirements are stable and fully predictable upfront.	System requirements evolve and emerge through iterative feedback.
Process Control Mode	Defined Process Control (Predictive).	Empirical Process Control (Adaptive).
Primary Progress Metric	Milestone completion and approved design documents.	Fully integrated, working, and testable software increments.

Cost of Change	Exponentially rises as the project progresses through phases.	Kept low through modular design, refactoring, and continuous integration.
Stakeholder Involvement	Heavy during requirements gathering and final user acceptance testing.	Continuous involvement across every delivery iteration.
Risk Profile	High late-stage operational, integration, and market-fit risk.	Low risk; early failure and fast feedback lower overall impact.

2.1 Empirical Process Control in Scrum

Scrum operationalizes empirical process control rather than relying on top-down, predictive planning. Empiricism asserts that knowledge stems from experience and that decision-making must be anchored on observed reality rather than theoretical projections.



The Scrum framework is structured around three foundational pillars:

- **Transparency:** Significant aspects of the process must be visible to those responsible for its outcome. Operational artifacts, shared definitions, and delivery metrics must be visible and standardized so all stakeholders share a common understanding of system state.
- **Inspection:** Scrum artifacts and progress toward sprint goals must be inspected regularly and thoroughly. Frequent inspection detects unexpected variances before they cause significant project drift.
- **Adaptation:** If an inspection reveals that one or more aspects of a process deviate outside acceptable limits, the process or material being processed must be adjusted immediately. Adaptation requires self-organizing teams capable of shifting tactical direction as soon as new data emerges.

2.2 The Three Scrum Roles

Scrum replaces traditional hierarchical management structures with three explicit, complementary roles. Together, these roles ensure accountability for product value, process execution, and technical delivery.

Scrum Team Accountabilities

Product Owner	Scrum Master	Developer
What to build & why	How to optimize flow	How to build & deliver

The Product Owner (PO)

The Product Owner is a single individual, not a committee, accountable for maximizing the business value of the product resulting from the work of the Scrum Team.

- **Core Responsibilities:**
  - Expressing and ordering Product Backlog items cleanly to best achieve goals and missions.
  - Optimizing the value of the work the Development Team performs.
  - Ensuring the Product Backlog is transparent, visible, and understood by all stakeholders.
  - Deciding when to release increments to production based on market readiness.
- **Anti-Patterns:** Serving as a pass-through proxy for management, delegating backlog prioritization to committee votes, or overriding technical implementation choices.

The Scrum Master (SM)

The Scrum Master acts as a servant-leader for the Scrum Team, ensuring that Scrum theories, practices, rules, and values are understood and implemented correctly.

- **Core Responsibilities:**
  - Facilitating Scrum events as needed or requested.
  - Coaching the Development Team in self-organization, cross-functionality, and technical focus.
  - Removing operational impediments (blockers) that hinder the team's progress.
  - Guiding the Product Owner in effective Product Backlog management and empirical planning.
- **Anti-Patterns:** Functioning as a traditional project manager, assigning tasks directly to developers, or running daily meetings as status update sessions.

The Development Team

The Development Team consists of professionals who do the work of delivering a usable, testable Increment of product at the end of each Sprint.

- **Key Attributes:**
  - **Self-Organizing:** The team determines *how* to turn Product Backlog items into working functionality without external direction.

- **Cross-Functional:** Possesses all the technical skills (design, development, testing, infrastructure) needed to create a releasable increment.
- **No Titles:** Every member carries the unified title of Developer, regardless of specialized skill sets.
- **Optimal Sizing:** Typically 3 to 9 members. Teams smaller than 3 risk skill gaps; teams larger than 9 face excessive internal communication overhead.

2.3 The Four Scrum Ceremonies (Events)

Scrum prescribes four timeboxed events designed to enable inspection and adaptation while minimizing unnecessary status meetings. Every event serves as a structured opportunity for empirical process control.



Source: cloudwards.net

1. Sprint Planning

- **Timebox:** Maximum 8 hours for a one-month Sprint (proportionately shorter for shorter Sprints).
- **Inputs:** Sprint Goal, product Backlog, latest product Increment, projected team capacity, and past performance metrics.
- **Outputs:** An explicit Sprint Backlog.
- **Mechanics:** Conducted in two main parts:
    1. *What can be done?* The Product Owner presents candidate backlog items, and the team selects what can be accomplished.
    2. *How will the work get done?* The Development Team negotiates technical execution and decomposes selected items into actionable tasks.

2. Sprint Execution & Daily Scrum

- **Sprint Execution:** The ongoing implementation phase where the Development Team self-organizes to turn Sprint Backlog items into a usable, testable Increment satisfying the Definition of Done
- **Daily Scrum Timebox:** 15 minutes, held at the same time and location every working day throughout Sprint execution period.
- **Participants:** Development Team (Scrum Master and Product Owner participate if they are actively building items).
- **Daily Scrum Mechanics:** The team inspects progress toward the Sprint Goal and evaluates how their trajectory is trending. Members answer three core operational questions:
    1. *What did I do yesterday that helped the Scrum Team meet the Sprint Goal?*
    2. *What will I do today to help the Scrum Team meet the Sprint Goal?*
    3. *Do I see any impediment that prevents me or the team from meeting the Sprint Goal?*

3. Sprint Review

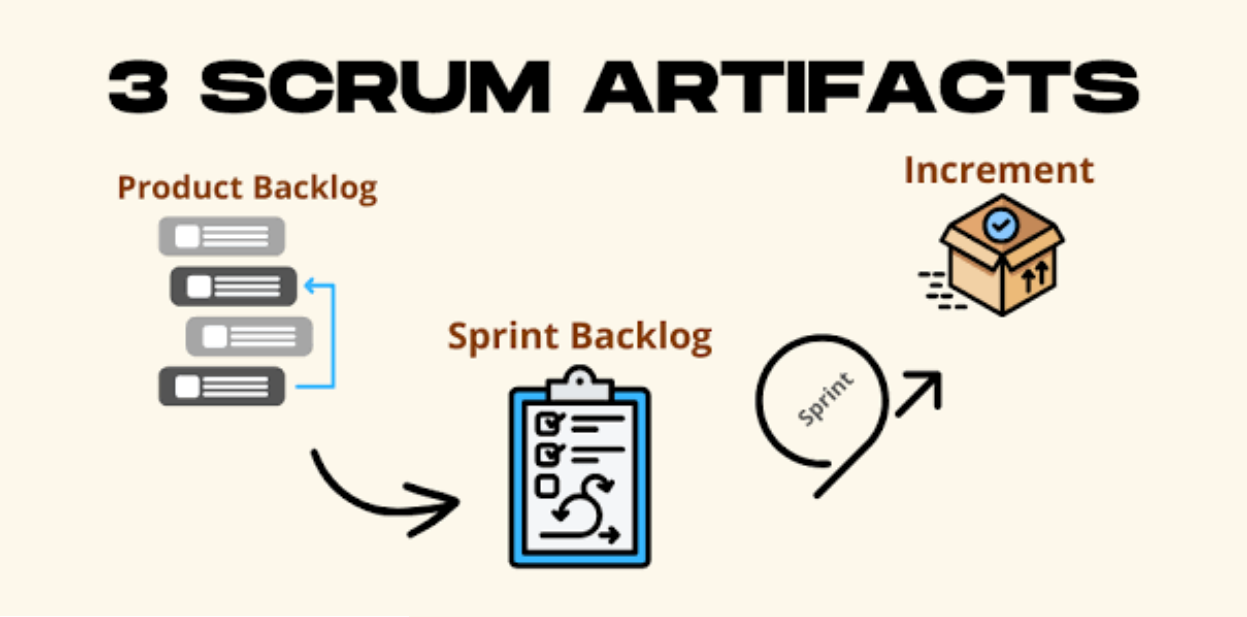
- **Timebox:** Maximum 4 hours for a one-month Sprint.
- **Participants:** Scrum Team and key business stakeholders invited by the Product Owner.
- **Mechanics:** An informal working session to inspect the usable Increment produced during the Sprint. The team presents completed, functional capabilities to gather direct feedback, assess market shifts, and adapt the Product Backlog for upcoming Sprints.

4. Sprint Retrospective

- **Timebox:** Maximum 3 hours for a one-month Sprint.
- **Participants:** Scrum Team (PO, SM, Development Team).
- **Mechanics:** An internal team inspection focused on continuous process improvement. The team inspects how the last Sprint went regarding people, relationships, process, and tools. Out of this meeting comes at least one actionable operational improvement to implement in the next Sprint.

2.4 The Three Scrum Artifacts

Scrum artifacts represent work or value, designed to optimize transparency of key information so that all participants inspect and adapt based on identical reference points.



Source: cloudwards.net

1. Product Backlog

- **Definition:** An ordered list of everything that is known to be required in the product. It is the sole source of requirements for any changes to be made to the product.
- **Characteristics:** It is dynamic and constantly evolving; as long as a product exists, its Product Backlog exists.
- **Refinement:** The ongoing process in which the Product Owner and Development Team add detail, estimates, and order to items in the backlog.

2. Sprint Backlog

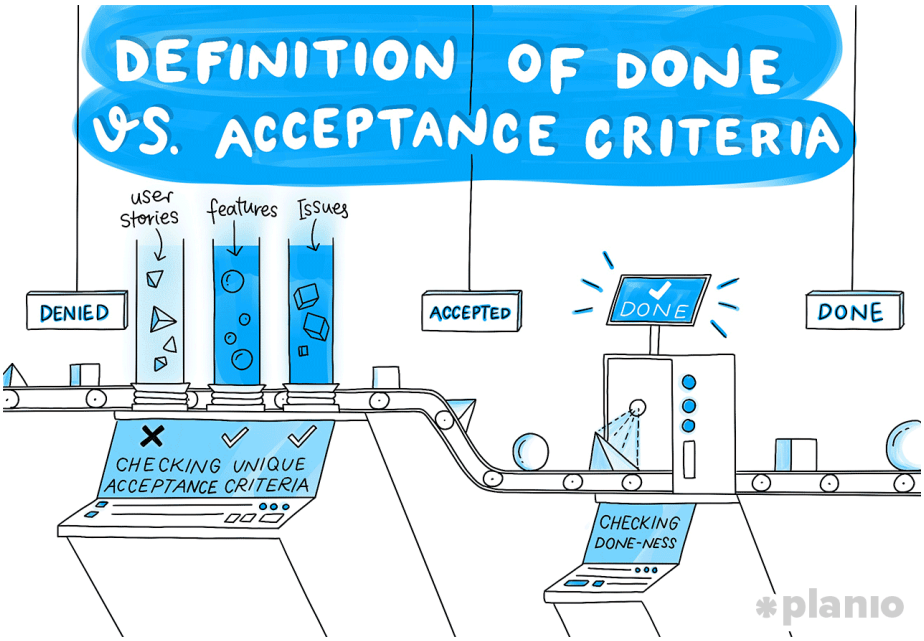
- **Definition:** The set of Product Backlog items selected for the Sprint, plus a practical plan for delivering the product Increment and realizing the Sprint Goal.
- **Characteristics:** Highly dynamic during the Sprint. As the Development Team learns more about the implementation details, they add, update, or remove technical tasks throughout the process.

3. Increment

- **Definition:** The sum of all the Product Backlog items completed during a Sprint and the value of the increments delivered in all previous Sprints.
- **Characteristics:** Must be in usable condition regardless of whether the Product Owner decides to release it to production immediately.

2.5 The Definition of Done (DoD)

The **Definition of Done (DoD)** is a formal, shared understanding among the Scrum Team of what quality standards code must meet before an item is considered a completed product Increment.



Source: plan.io

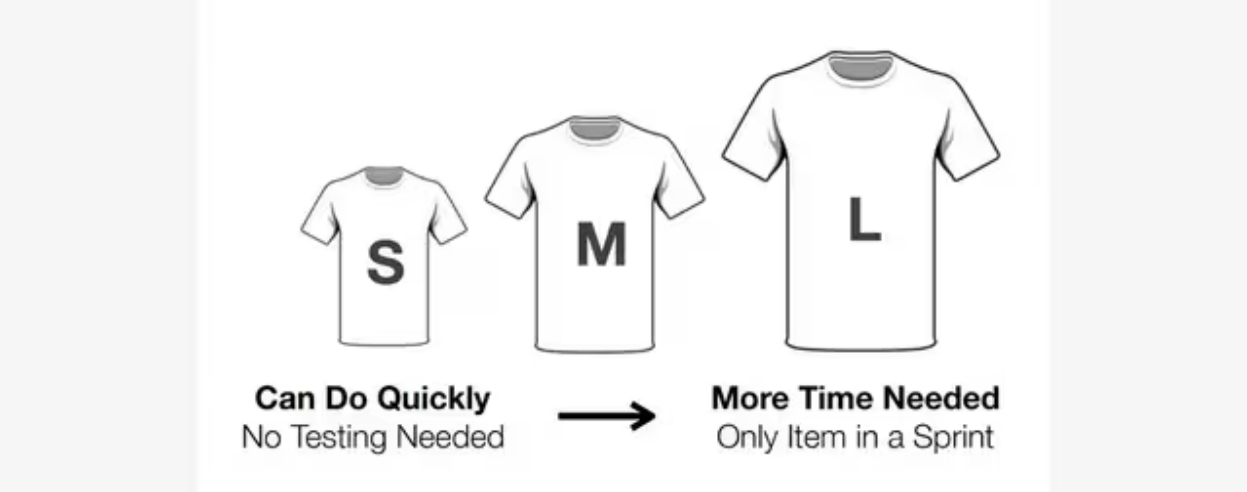
- **Purpose:** Ensures absolute transparency so that when an item is declared "Done," everyone understands what was completed and what quality measures were validated.
- **DoD vs. Acceptance Criteria:**
  - **Acceptance Criteria:** Unique to each individual Product Backlog item (e.g., *"User can reset password via SMS confirmation"*).
  - **Definition of Done:** Applies uniformly to *all* backlog items (e.g., *"Unit test coverage > 80%, peer reviewed, and merged into main line"*).
- **Impact of Incomplete DoD:** If an item does not satisfy the Definition of Done, it cannot be demonstrated at the Sprint Review or released; it must return to the Product Backlog for future Sprint planning.

3.1 Relative Sizing vs. Absolute Estimation

Traditional software project management relies on absolute time estimation—predicting tasks in exact hours or days. Absolute estimation fails in dynamic software environments because:

- **Individual Skill Variance:** A senior engineer might complete an architectural task in 4 hours, whereas a junior engineer might require 24 hours for the same task.
- **The Anchoring Bias:** When developers are forced to commit to exact hourly figures early in a project, those estimates are often treated as rigid deadlines, leading to compromised code quality or bloated buffer times.
- **Cognitive Complexity Overhead:** Estimating low-level hourly tasks far in advance consumes significant energy while offering minimal precision due to unforeseen technical complications.

Agile frameworks replace absolute estimation with **Relative Sizing**. Instead of asking "How many hours will this take?", relative sizing asks "How big is this task compared to a known baseline task?"



Source: knowledgehut.com

Relative estimation decouples the effort and complexity of a requirement from the time required to complete it. Time becomes a variable dependent on the team's historical execution capacity (Velocity).

3.2 Story Points and the Abstract Sizing Vector

A **Story Point** is an abstract, unitless metric that encapsulates three distinct dimensions of software work:

1. **Complexity:** How difficult is the implementation? (e.g., algorithmic difficulty, data transformations, multi-system integrations).
2. **Effort:** How much sheer volume of work is required? (e.g., writing repetitive UI boilerplate vs. modifying a single configuration file).
3. **Risk and Uncertainty:** How many unknown variables exist? (e.g., unproven third-party APIs, legacy refactoring, ambiguous legacy database schemas).

The Modified Fibonacci Sequence

Agile teams express Story Points using a modified Fibonacci scale:

Scale = {1, 2, 3, 5, 8, 13, 20, 40, 100}

The non-linear growth of the scale reflects the increasing uncertainty that comes with larger tasks. Small tasks (1, 2, 3) are well-understood and low-risk. Larger tasks (20, 40, 100) are too complex or vague to estimate accurately; they must be broken down into smaller, manageable sub-components during Product Backlog refinement.

3.3 Consensus-Based Estimation: Planning Poker

Planning Poker is a collaborative, consensus-based estimation technique designed to eliminate cognitive bias, mitigate peer influence, and encourage open technical debate.

Step-by-Step Execution Protocol

1. **Presentation:** The Product Owner introduces a Product Backlog item (PBI). The team clarifies functional boundaries and acceptance criteria.
2. **Technical Discussion:** Developers debate potential implementation approaches, dependencies, and architectural impacts.
3. **Private Valuation:** Each developer selects a card from their Fibonacci deck corresponding to their individual estimate. No discussion is permitted during card selection to prevent **anchoring bias** (where a senior engineer's estimate sways the rest of the team).
4. **Simultaneous Reveal:** All participants flip their cards at the exact same moment.
5. **Divergence Resolution:** If the estimates match, the consensus score is recorded. If there is significant variance (e.g., estimates of 2, 3, 3, and 13), the team does *not* average the numbers. Instead:
    - The **lowest estimator** explains their reasoning (e.g., *"I know an existing helper class that makes this trivial"*).
    - The **highest estimator** explains their reasoning (e.g., *"This endpoint requires updating three downstream schemas that aren't documented"*).
6. **Convergence Iteration:** The team re-votes until they reach reasonable consensus on a unified estimate.

3.4 Velocity and Mathematical Forecasting

Velocity is a metric measuring the amount of work a Scrum Team completes during a single Sprint, expressed as the sum of Story Points associated with items that meet the formal **Definition of Done**.

$$\text{Sprint Velocity} = \frac{\text{Completed Story Points (DoD Compliant)}}{}$$

Single Sprint

Important Velocity Rules

- **Binary Counting:** Story points are counted on an all-or-nothing basis. If an 8-point story is 90% complete at the end of a Sprint, it contributes **0 points** to that Sprint's velocity. The item remains in the backlog, and points are counted only when it is fully completed in a future Sprint.
- **Team-Specific Metric:** Velocity is a unique metric tied to a specific team's capacity, culture, and relative baseline. **Velocity cannot be used to compare performance between different teams.** A team delivering 50 points per Sprint is not inherently more productive than a team delivering 20 points, as their underlying point baselines differ.

Forecasting Work with Mean Velocity

Once a team establishes a stable historical baseline over 3 to 5 Sprints, velocity enables empirical forecasting:

$$\bar{V} = \frac{\sum_{i=1}^{N} V_i}{N}$$

Where $\bar{V}$ is the mean velocity across N observed Sprints.

If a Product Backlog contains 150 total Story Points and the team maintains a mean velocity ($\bar{V}$) of 25 points/Sprint, the expected remaining duration to release is calculated as:

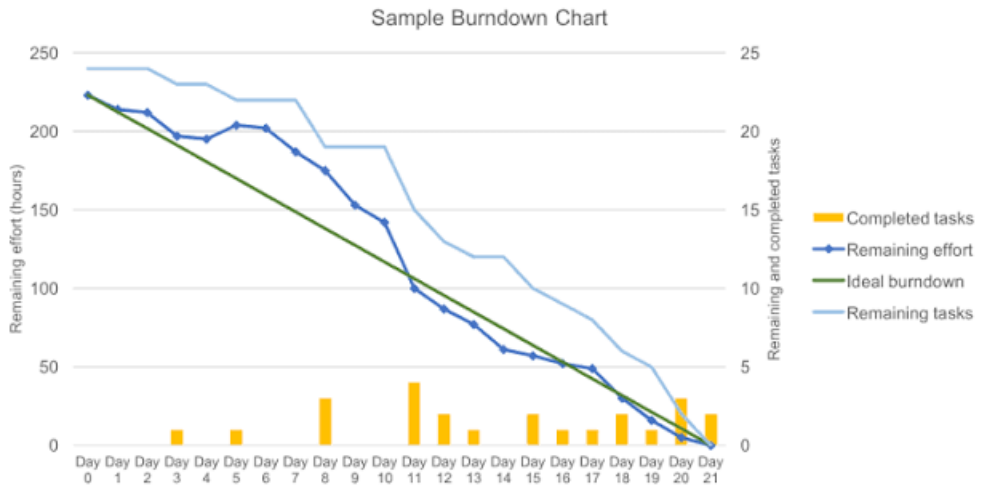
$$\text{Estimated Sprints to Release} = \frac{\text{Total Backlog Points}}{\bar{V}} = \frac{150}{25} = 6 \text{ Sprints}$$

3.5 Burndown Visual Analytics

Visual tracking charts convert raw team performance data into clear, actionable progress indicators.

Sprint Burndown Chart

Tracks the remaining work within a single Sprint on a daily basis.



Source: Wikipedia

- **Vertical Axis:** Total Story Points or tasks remaining in the Sprint Backlog.
- **Horizontal Axis:** Working days in the Sprint.
- **Ideal Line:** A linear baseline connecting total committed points at Day 0 to 0 points on the final day.
- **Actual Line:** Plotted daily during the Daily Scrum.
    - *Above Ideal Line:* Team is running behind schedule (potential capacity issue or hidden complexity).
    - *Below Ideal Line:* Team is completing work faster than planned (available capacity to pull in backlogged items).